Boggle Board

AlgoExpert · Hard · Graphs

The problem

- Which words can be formed by chaining 8-directionally adjacent letters without reusing a cell?
- Trie of words + DFS from every cell.

Intuition

- A trie of the words lets the board DFS prune paths that can't spell any word.

Brute force vs optimal

- Search each word separately on the board → O(words · cells · 8^L)
- Trie + single DFS from each cell → prunes shared prefixes
- Mark cells used on the current path

Approach

- 1. Insert words into a trie
- 2. DFS from every cell along trie edges
- 3. Mark visited cells
- 4. Record words at word-end markers

Worked example

t	h	i	s
w	a	t	s
o	a	h	g
f	g	d	t

```
words: this, two, fat, that…
```

Board and dictionary (built into a trie)

Worked example

t	h	i	s
w	a	t	s
o	a	h	g
f	g	d	t

```
t → trie has 't' ✓
```

Start at 't'

Worked example

t	h	i	s
w	a	t	s
o	a	h	g
f	g	d	t

```
t-h-i-s → word 'this' ✓
```

t → h → i → s: found 'this'

Worked example

t	h	i	s
w	a	t	s
o	a	h	g
f	g	d	t

```
t → w: trie has 'tw' ✓ (two)
t → a: … prune if no branch
```

Prune when the trie has no branch

Java solution (1/2)

```java
public class BoggleBoard {
    private static class Trie { Map<Character, Trie> next = new HashMap<>(); String word; }

    public static List<String> boggleBoard(char[][] board, String[] words) {
        Trie root = new Trie();
        for (String w : words) {
            Trie t = root;
            for (char c : w.toCharArray()) t = t.next.computeIfAbsent(c, k -> new Trie());
            t.word = w;
        }
        Set<String> found = new LinkedHashSet<>();
        boolean[][] used = new boolean[board.length][board[0].length];
        for (int r = 0; r < board.length; r++) for (int c = 0; c < board[0].length; c++) dfs(board, r, c, root, used, found);
        return new ArrayList<>(found);
    }
```

Java solution (2/2)

```
    private static void dfs(char[][] b, int r, int c, Trie t, boolean[][] used, Set<String> found) {
        if (r < 0 || c < 0 || r >= b.length || c >= b[0].length || used[r][c]) return;
        Trie nx = t.next.get(b[r][c]);
        if (nx == null) return;
        if (nx.word != null) found.add(nx.word);
        used[r][c] = true;
        for (int dr = -1; dr <= 1; dr++) for (int dc = -1; dc <= 1; dc++) if (dr != 0 || dc != 0) dfs(b, r + dr, c + dc, nx, used, found);
        used[r][c] = false;
    }
}
```

Complexity

- Time: O(ws + w*h*8^s)
- Space: O(ws + w*h)

Edge cases & pitfalls

- 8-directional moves
- Don't reuse a cell in one word
- Dedupe found words

Interview tips & follow-ups

- Explain why the trie prunes
- Follow-up: remove found words from the trie

Recap

- A trie of the words lets the board DFS prune paths that can't spell any word.
- Complexity: O(ws + w*h*8^s)